

Building AI agents from scratch

The model asks for a function call, your code runs it, and the result goes back in, until the model can answer.

THE QUESTION

A language model can only write text. How does it check today's weather, multiply 4729 by 8314 exactly, or search your notes?

THE INTUITION

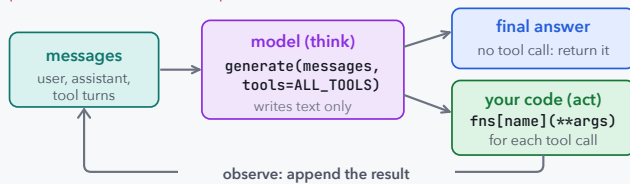
Give it a menu of tools as JSON schemas. It replies with a request such as `calculate("4729 * 8314")`; your code runs it, appends the result and asks again, until the model answers in plain text.

KEY TAKEAWAY

Agent = LLM + tools + loop.
The model decides and your code executes, so your code checks every call, caps the steps and never passes model text to `eval()`.

1 An agent is an LLM, tools and a loop

for step in 1 to max_steps (5 in the notebook):
past the limit: "Reached max steps without a final answer."



Asked for the weather, a plain LLM can only guess from its training data; with a tool it checks. Coding assistants that read files, run tests and edit code run this same loop with bigger tools.

2 A tool is a function plus a schema

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get the current weather ... for a city. Available cities: Gandhinagar, Mumbai, ..., New York.",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string"}
      },
      "required": ["city"]
    }
  }
}
```

what the model writes back;
a key in TOOL_FUNCTIONS

all the model reads to choose:
when to call, valid inputs

JSON Schema: one typed entry
per argument

the model must fill these

the lab's weather_tool, description shortened · the model never sees the function body

```
def get_weather(city: str) -> str:
    data = {"mumbai": {"temp_c": 32,
                        "condition": "Humid", "humidity": 80},
            ...} # 8 cities, mock
    out = data.get(city.lower(),
                  {"error": f"No data for {city}"})
    return json.dumps(out)
```

Return a string (JSON is fine) and report problems as text the model can read.

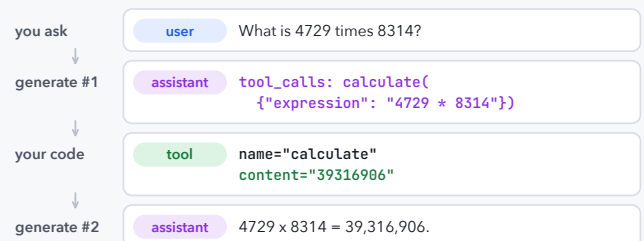
3 The chat template adds the menu

```
parsed = processor.parse_response(raw)
{"content": "",
 "tool_calls": [{"function": {
  "name": "calculate",
  "arguments": {"expression": "4729 * 8314"}}}]}
```

OpenAI-style APIs send arguments as a JSON string: `json.loads` it first

`apply_chat_template(messages, tools=...)` puts the schemas in the prompt; `model.generate` runs, and `parse_response` reads the new tokens. The notebook runs Gemma 4 E2B in 4-bit NF4 on a free Colab T4 (about 1.5 GB). Hosted APIs: see the LLM APIs sheet.

4 One tool call takes two model calls



2 model calls, 1 tool call · the product is computed; the final wording is illustrative

Append the assistant turn holding `tool_calls` and a `tool` turn with the result, then generate again. Each content is a list of text parts.

5 The model picks a tool, or none

question, then the call the lecture expects	the tool returns
What's 17 squared? <code>calculate("17 ** 2")</code>	289
Is it raining in Mumbai? <code>get_weather("Mumbai")</code>	Humid, 32 °C
Convert 100°F to Celsius <code>convert_units(100, "fahrenheit", "celsius")</code>	37.78 celsius
When did we learn about Docker? <code>search_notes("docker")</code>	containerization, ...
What is the capital of France? no tool call	answers from memory

results from the notebook's tools, run here · no model was called

`search_notes` over a dict is a tiny RAG: retrieve, then the model writes.

6 The loop is about ten lines

```
def agent(q, tools=ALL_TOOLS,
          fns=TOOL_FUNCTIONS, max_steps=5):
    messages = [user(q)]
    for step in range(1, max_steps + 1):
        r = generate(messages, tools=tools)
        calls = r["parsed"].get("tool_calls")
        if not calls:
            return r["text"] # done
        messages.append(assistant(calls))
        for tc in calls:
            name = tc["function"]["name"]
            args = tc["function"]["arguments"]
            if name in fns:
                out = fns[name](**args)
            else:
                out = f"Error: unknown tool '{name}'"
            messages.append(tool(name, str(out)))
    return ("Reached max steps without "
           "a final answer.")
```

user, assistant and tool build panel 4's dicts. A new tool is one more schema and one more dict entry.

Cap the loop and check every call

A multi-step trace, the cost of an endless loop, why `eval()` is unsafe, and guards for bad calls.

7 Four tool calls, three tools, one answer

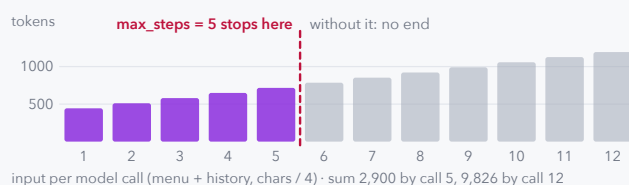
```
max_steps = 5
```

step 1	step 2	step 3	step 4	step 5	
1	I need sqrt(144) <code>calculate("144 ** 0.5")</code>				12.0
2	I need Mumbai's temperature <code>get_weather("Mumbai")</code>				temp_c 32
3	convert 32 °C to °F <code>convert_units(32, "celsius", "fahrenheit")</code>				89.60 fahrenheit
4	add them <code>calculate("12.0 + 89.6")</code>				101.6
5	no tool needed The answer is 101.6.				

4 tool calls, 3 tools, 5 model calls · messages grow 1, 3, 5, 7, 9 · scripted model, real tools

The lecture's question: the square root of 144 plus Mumbai's temperature in Fahrenheit. The 32 in step 3 comes from step 2's result. With `max_steps=5` there was no room for a sixth model call.

8 Each step re-sends the whole history



A model stuck on `get_weather("Narnia")` never stops by itself. Every call re-reads the menu and all past turns, so the total grows faster than the step count. Cap steps, spend and request rate.

9 eval() runs whatever the model writes

same input, three calculators	char filter + eval()	eval(), no builtins, np	AST walk (next panel)
4729 * 8314	39316906	39316906	39316906
sqrt(144) + log(e)	refused	13.0	13.0
np.mean([3, 5, 7, 11, 13])	refused	7.8	7.8
9 ** 9 ** 9	hangs	hangs	refused
__import__('os').getcwd()	refused	NameError	refused
().__class__.__base__ .__subclasses__()	refused	runs: 383 classes	refused
np.loadtxt('notes.txt', 'U8')	refused	runs: reads the file	refused

9 ** 9 ** 9 has 369,693,100 digits (not run) · amber: refuses a fair question
rose: does something it should not · `getcwd()` stands in for the notebook's `os.system(...)`

The notebook's filter allows only digits and `+*/()`, so it refuses the `sqrt()` its own schema advertises, yet lets `9 ** 9 ** 9` through. Emptying `__builtins__` still leaves `np` and object attributes.

12 Pause and think

No `max_steps`? A stuck model never stops.
Real weather? Swap the dict for an HTTP call.
Share tools? MCP: one tool server, many models.

10 Walk the syntax tree instead

```
OPS = {ast.Add: op.add, ast.Sub: op.sub,
       ast.Mult: op.mul, ast.Div: op.truediv,
       ast.Pow: op.pow, ast.MatMult: op.matmul,
       ast.USub: op.neg}
NAMES = {"pi": math.pi, "e": math.e}
FUNCS = {"sqrt": math.sqrt, "log": math.log,
         "np.mean": np.mean, "np.dot": np.dot}
def ev(n):
    # one rule per node
    match n:
        case ast.Constant(value=int() | float()):
            return n.value
        case ast.List():
            return [ev(x) for x in n.elts]
        case ast.Name():
            return NAMES[n.id] # or KeyError
        case ast.UnaryOp():
            return OPS[type(n.op)](ev(n.operand))
        case ast.BinOp():
            a, b = ev(n.left), ev(n.right)
            if type(n.op) is ast.Pow and b > 100:
                raise ValueError("exponent too big")
            return OPS[type(n.op)](a, b)
        case ast.Call(keywords=[]):
            f = FUNCS[ast.unparse(n.func)]
            return f(*[ev(a) for a in n.args])
            raise ValueError(type(n).__name__)
```

Run `ev(ast.parse(expression, mode="eval").body)` in a try. Add `sin`, `factorial`, `np.array` and the exercise gives 13.0, 120.0, 7.8, 32, `[[19, 22], [43, 50]]`. Cap factorial; time out slow tools.

11 Turn bad calls into error messages

problem · guard	bad call (scripted), then what comes back
unknown tool name check it is in the dict	<code>get_wether(city="Mumbai")</code> Error: unknown tool 'get_wether'
wrong argument name catch the TypeError	<code>get_weather(town="Mumbai")</code> Error: ... unexpected keyword argument 'town'
broken JSON arguments json.loads in the try	<code>get_weather({'city': "Mumbai"})</code> Error: Expecting ',' delimiter ...
a dangerous tool ask the user first	<code>delete_file(path="data/")</code> Error: user declined

```
def run_tool(name, args, fns):
    if name not in fns:
        return f"Error: unknown tool '{name}'"
    if name in DANGEROUS and \
        not ask_user(name, args):
        return "Error: user declined"
    try:
        if isinstance(args, str): # JSON text
            args = json.loads(args)
        return str(fns[name](**args))
    except Exception as e: # bad JSON or args
        return f"Error: {e}"
```

Errors go back as `tool` turns, so the model can retry.

LLM + tools + loop · the schema is the menu · the model decides, your code executes · `max_steps` · no `eval()`

Source: lecture "Building AI agents from scratch" and its notebook, with the NumPy-aware calculator exercise, nipunbatra.github.io/stt-ai-teaching. Code follows the notebook (transformers, Gemma 4 E2B, OpenAI-style tool schemas); the message helpers, `run_tool` and the AST calculator are added here. No model was run: a scripted stand-in model drives the notebook's real tools and loop, so the model's wording is illustrative; tokens are characters / 4. This is the last sheet in the series, which starts with the HTTP and APIs sheet.